

VI. ВЫВОДЫ И ЗАКЛЮЧЕНИЕ

Описанный в работе макет стенда-тренажера может быть использован как в учебном процессе подготовки и переподготовки обучающихся по направлению «Информационная безопасность», а также для тренинга по пентесту таких технических специалистов в сфере информационных технологий как системные и сетевые администраторы; разработчики веб-приложений; инженеры-программисты; архитекторы безопасности, аналитики и инженеры; консультанты по информационной безопасности. Работа со стендом позволяет обеспечить действия тестеров на проникновение для различных ситуаций.

При дальнейшем развитии структуры макета следует учесть, что многие уязвимые машины имеют нестандартные конфигурации. Простая настройка инструментов атаки на цели и нажатие кнопки «далее» во многих случаях может и не сработать. Тестеры на проникновение должны хорошо разбираться в уязвимостях проектируемых или эксплуатируемых автоматизированных информационных систем, чтобы выполнить свою задачу по обеспечению защищенности доверенных объектов, а не полагаться только на предлагаемые программные и программно-аппаратные инструменты аудита информационной безопасности.

СПИСОК ЛИТЕРАТУРЫ

1. Мещеряков Р. В., Шелупанов А. А. Концептуальные вопросы информационной безопасности региона и подготовки кадров // Труды СПИИРАН. 2014. № 3 (34). С. 136–159.
2. Скабцов Н. В. Аудит безопасности информационных систем. СПб.: Питер, 2018. 272 с.
3. Weidman G. Penetration Testing: A Hands-On Introduction to Hacking. NY.: Press. Inc, 2014. 478 с.
4. Dorofeev A. V., Rautkin Y. V. Applied Aspects of Security Testing // CEUR Workshop Proceedings. Selected Papers of the VIII All-Russian Scientific and Technical Conference on Secure Information Technologies, BIT 2017. 2017. Vol. 2081. P. 49–53.
5. Barabanov A. V., Lavrov A. I., Markov A. S., Polotnyanshikov I. A., Tsirlov V. L. The study into cross-site request forgery attacks within the framework of analysis of software vulnerabilities // Trudy ISP RAN/Proc. ISP RAS. 2017. Vol. 29, Is. 5. P. 7–18.
6. Итоги внешних пентестов. URL : <https://www.ptsecurity.com/analytics/Cobalt-Snatch-rus.pdf>, свободный (дата обращения 20.05.2020).
7. Karadoğan, Daş R. Analysis of attack types on TCP/IP based networks via exploiting protocols // 23rd Signal Processing and Communications Applications Conference (SIU), Malatya, 2015.
8. Цветкова О. Л., Айдинян А. Р. Интеллектуальная система оценки информационной безопасности предприятия от внутренних угроз // Вестник компьютерных и информационных технологий. 2014. № 8 (122). С. 48–53.

УДК 004.056.53

ПРИМЕНЕНИЕ ИСКУССТВЕННОЙ НЕЙРОННОЙ СЕТИ ПРИ ПОСТРОЕНИИ МОДЕЛИ ДЛЯ РАСПОЗНАВАНИЯ ВРЕДОНОСНОГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

APPLICATION OF AN ARTIFICIAL NEURAL NETWORK AT CREATION OF MODEL FOR RECOGNITION OF THE MALICIOUS SOFTWARE

О. Т. Данилова, А. П. Киреев

Омский государственный технический университет, г. Омск, Россия

O. T. Danilova, A. P. Kireev

Omsk State Technical University, Omsk, Russia

Аннотация. В настоящей работе предлагается подход к построению модели обнаружения программ удаленного управления с измененным функционалом с использованием технологии анализа поведения легального и вредоносного программного обеспечения (ПО). Классификация процессов на основе их поведения производится с использованием искусственной нейронной сети. Нейронная сеть состоит из 34 нейронов на входном слое, 6 нейронов на скрытом слое и 1 выходного нейрона. Функция активации нейронов – сигмоидальная. Для подстройки весовых коэффициентов нейронной сети применяется алгоритм обучения с учителем – алгоритм обратного распространения ошибки. Выбор данной архитектуры обусловлен балансом между эффективностью классификации и вычислительной нагрузкой, которую создает сеть на компьютерную систему в режиме распознавания. В ходе проведения исследования была разработана модель системы распознавания программного обеспечения, имеющего в своем составе функционал, предназначенный для удаленного управления, на основе искусственной нейронной сети.

При решении поставленной задачи для формирования признакового пространства была проведена оценка распределения событий в компьютерной системе по группам для различных классов программного обеспечения. В процессе верификации разработанной аналитической модели и оценки эффективности полученного решения были поставлены эксперименты для обоснования выбора количества эпох обучения; минимального объема обучающей выборки; минимальной длины анализируемой цепочки событий компьютерной системы.

Ключевые слова: программы удаленного управления, нейронная сеть, индикаторы компрометации, системы распознавания.

DOI: 10.25206/2310-9793-8-2-97-104

I. ВВЕДЕНИЕ

Программы удаленного управления (ПУУ), используемые для удаленной технической поддержки и администрирования, позволяют дистанционно управлять компьютером через сеть Интернет или в локальной сети. Данные программы могут захватывать и преобразовывать изображения на удаленном компьютере и отсылать их на локальный компьютер. Такие действия как нажатие клавиш на клавиатуре или движения мыши также могут быть переданы на удаленный компьютер, который интерпретирует их как сигналы, введенные непосредственно пользователем этого компьютером. Такое программное обеспечение относится к потенциально опасному (*riskware*), т.е. в случае злоумышленного использования может причинить вред пользователю: вызвать блокирование, уничтожение или модификацию информации, а также нарушить работу компьютера или компьютерных сетей. Особенностью использования легальных ПУУ является то, что такие программы не обнаруживаются антивирусными детекторами, а единственная задача злоумышленников при использовании этих приложений в своих целях – это обеспечение скрытности их установки и функционирования [1, 2].

Детектирование атак основывается на различных алгоритмах, таких как интеллектуальный анализ данных с применением статистических методов [3, 4] или методов классификации [5, 6]; методов опорных векторов; методов с применением искусственных нейронных сетей, обучающихся с учителем или без учителя [7–12]. В методах обучения с учителем применяется набор пар образцов «входные данные – ожидаемый ответ» для корректировки весовых значений нейронов в нейронных сетях, а методы обучения без учителя позволяют распознать аномальное поведение с учетом приобретенных в прошлом знаний.

В представляемой работе обнаружение программ удаленного управления с измененным функционалом предлагается проводить с использованием модели, основанной на искусственной нейронной сети, с реализацией способов использования индикаторов компрометации.

II. ПОСТАНОВКА ЗАДАЧИ

Для определения местоположения следов активности ПУУ используется модель Cyber Kill Chain, в которой обнаружение признаков вредоносной активности в инфраструктуре осуществляется с использованием дискретных данных – индикаторов компрометации IOC (*indicator of compromise*). Термин Cyber Kill Chain изначально был предложен корпорацией Lockheed Martin как часть их модели Intelligence Driven Defense для идентификации и предотвращения процессов кибер-вторжений [13]. Данная модель включает в себя последовательность из семи шагов, которые необходимо выполнить злоумышленнику для того, чтобы достичь своих целей, атакуя сеть, извлекая данные и поддерживая присутствие в организации. Поэтому блокировка злоумышленника на любом этапе разрывает всю цепочку атаки.

В данной работе учтено, что применение данной модели становится малоэффективным в случаях использования:

- 1) ранее неизвестных или модифицированных приложений для осуществления удаленного управления;
- 2) совокупности приложений, реализующих удаленное управление совместно, при этом каждое из приложений обладает своим, совершенно легитимным функционалом.

Исходя из вышеизложенного, в исследовании предлагается расширение модели Cyber Kill Chain за счет использования технологии анализа поведения легального и вредоносного программного обеспечения (ПО).

III. ТЕОРИЯ

Формальная постановка задачи обнаружения программного обеспечения имеющего в своем составе функционал для удаленного управления, может быть представлена следующим выражением:

$$F: X \rightarrow Y,$$

где X – множество всех программ; $Y = \{C, V\}$ – множество индексов классов программ; C – индекс класса программ, не имеющих функциональных возможностей для удаленного управления; V – индекс класса программ

предназначенных для удаленного управления; F – индикаторная функция, отображающая элементы множества X в индексы классов C и V [7].

Очевидным решением данной задачи является построение функции отображения:

$$F(x) = \begin{cases} \min_x P_{fpr}(v \in C), X = C \cup V \\ P_{fpr}(c \in V) \leq P_{fpr_{max}} \end{cases},$$

здесь $v \in V$ – программа с достоверным знанием о наличии в ней функционала удаленного управления; $c \in C$ – программа с достоверным знанием об отсутствии в ней такого функционала; P_{fpr} – вероятность ошибки первого рода; P_{fpr} – вероятность ошибки второго рода.

Нейронная сеть, применяемая для решения поставленной задачи, состоит из 34 нейронов на входном слое, 6 нейронов на скрытом слое и 1 выходного нейрона. Функция активации нейронов – сигмоидальная. Для подстройки весовых коэффициентов нейронной сети применяется алгоритм обучения с учителем – алгоритм обратного распространения ошибки. Выбор данной архитектуры обусловлен балансом между эффективностью классификации и вычислительной нагрузкой, которую создает сеть на компьютерную систему в режиме распознавания.

Для построения модели примем следующие допущения:

– под программным обеспечением, имеющим в своем составе функционал для удаленного управления, будем рассматривать только то программное обеспечение, которое работает в режиме удаленного рабочего стола, то есть удаленный пользователь как минимум видит экран компьютера, может управлять мышью и клавиатурой;

– зададим некоторый набор классов процессов, каждый из которых полностью отождествляется с совокупностью всех его возможных действий;

– каждый процесс может быть представлен динамической классификацией в виде модели черного ящика, на входе и на выходе, которого, предполагается фиксация сигнала при этом игнорируются действия, происходящие внутри;

– поведение процесса представим как связанные с ним последовательности инициированных событий;

– последовательность вызовов системных функций для одного и того же класса вредоносных программ не является четко определенной и носит вероятностный характер.

Пусть E – есть множество всех возможных событий в компьютерной системе. В системе функционирует N процессов, а E_i – есть множество событий i -го процесса, которые могут произойти в случае его активности. Большинство процессов не выполняет сразу всех действий, которые заложены для каждого из них, при этом для многих процессов многие действия схожи. Процесс может достаточно долгое время генерировать события только из некоторого подмножества E_i . Если в компьютерной системе существует процесс, для которого $E_j \subset E_i$, то j -ый процесс будет определен как i -ый.

Формально поведение каждого конкретного процесса в операционной системе может быть представлено в виде последовательности событий:

$$B_i = (a_1, a_2, \dots, a_n), a_i \in A_{PID}, 1 \leq i \leq n.$$

Выберем в качестве событий в компьютерной системе, описывающих поведение процесса, следующие:

- 1) вызов приложениями API функций операционной системы;
- 2) передачу данных между процессами приложений с использованием технологии межпроцессного взаимодействия (*Inter-Process Communication*);
- 3) передачу приложениями данных по компьютерной сети;
- 4) эмуляцию ввода с клавиатуры, мыши.

Поскольку процесс однозначно инициализируется исполняемым файлом, то он имеет определенный набор инструкций и не может выполнить никаких действий, которые в нем не заложены. Любая вставка (цепочки NOP) никак не может повлиять на функциональные возможности процесса. Такой подход позволяет полностью игнорировать любые изменения исполняемого файла, которые не несут за собой изменение поведения.

В качестве параметров события рассматриваются: тип события, идентификатор процесса (PID) вызвавшего событие, а также параметры, передаваемые в системные функции при вызове, например для функций работы с сетью: сетевой адрес сервера; тип передаваемых данных; объем передаваемых данных. Для функций ввода такими параметрами могут выступать идентификаторы устройств и т.д.

Множество программ S , для которых имеются априорные данные о наличии в них функционала, может быть разбито на два класса:

$$S = S_{rat} \cup S_{clir}.$$

Допустим, что для класса программ S_{clir} , имеется множество цепочек вызовов системных функций, характеризующих «чистые» приложения:

$$P^1 = \{P_1^1, P_2^1, P_3^1, \dots, P_{n1}^1\},$$

а для класса программ S_{rat} , имеется множество цепочек вызовов системных функций, характеризующих программное обеспечение, имеющее в своем составе функционал для удаленного управления:

$$P^2 = \{P_1^2, P_2^2, P_3^2, \dots, P_{n2}^2\},$$

здесь $n1$ – число «чистых» цепочек; $n2$ – число цепочек, полученных в результате выполнения функций программного обеспечения удаленного управления.

При этом $P = \{P^1, P^2\}$ представляет собой все множество рассматриваемых цепочек ($n = n1 + n2$) – общее число цепочек.

Каждая цепочка представляется последовательностью вызовов системных функций, заданных идентификаторами:

$$Api = \{Api_1, \dots, Api_a\},$$

где $a = 1 \dots A$, A – число существующих в операционной системе Win32 API функций.

Каждый i -й процесс визуализируется цепочкой из m вызовов системных функций, однозначно его характеризующих:

$$P_i = \{Api_1, \dots, Api_m\},$$

где $i = 1 \dots n$.

Поскольку каждая такая цепочка имеет неопределенную длину, то ее анализ затруднителен. Для исключения такого затруднения каждую цепочку P_i можно разбить на кадры определенной длины, из которых формируется множество объектов $\{X\}$, заданных векторами признаков $X = \{X_1, X_2, \dots, X_d\}$, поступающих на сенсоры входного слоя нейронной сети.

В общем случае задача обучения нейронной сети сводится к нахождению некоторой функциональной зависимости:

$$Y = F(X),$$

где X – входной вектор, Y – выходной вектор.

Выходной вектор нейронной сети на объекте X_i вычисляется как суперпозиция:

$$y^m(x_i) = f_m \left(\sum_{h=0}^H \omega_{hm} u^h \right);$$

$$u^h(x_i) = f_h \left(\sum_{j=0}^J \omega_{jh} u^j \right),$$

здесь $m = 1, \dots, M$ количество нейронов выходного слоя, с функцией активации f_m и выходами y^m ; $h = 1, \dots, H$ количество нейронов скрытого слоя с функцией активации f_h и выходами u^h .

Поскольку данная задача имеет бесконечное множество решений при ограниченном наборе входных данных, то при обучении для ограничения пространства поиска ставится задача минимизации целевой функции ошибки нейронной сети, которая находится согласно методу наименьших квадратов:

$$E(\omega) = \frac{1}{2} \sum_{m=1}^M (y_m - d_m)^2,$$

где y_m – значение m -го выхода нейронной сети; d_m – целевое значение m -го выхода; M – число нейронов в выходном слое.

Обучение нейронной сети осуществляется методом обратного распространения ошибки, то есть на каждой итерации изменение веса производится по формуле:

$$\Delta\omega_{ij} = -\eta \frac{\partial E}{\partial \omega_{ij}},$$

где η – параметр, определяющий скорость обучения.

Для того чтобы эффективно распознавать вредоносный код необходимо составить библиотеку событий, которыми являются программные действия, связанные с вызовами системных функций, приводящими к изменениям в системе. Составление такой библиотеки событий делается на основе анализа большого числа вредоносных программ и выделения в них часто встречающихся характерных фрагментов. При этом основной задачей является формирование наиболее полного набора значимых признаков.

IV. РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ

В ходе выполнения работы была разработана функциональная модель системы распознавания на основе искусственной нейронной сети программного обеспечения, имеющего в своем составе функционал, предназначенный для удаленного управления (рис. 1). Идея разработанного алгоритма заключается в реализации возможностей установления наличия или отсутствия программ, предназначенных для удаленного управления в исследуемой системе, а если априорные знания об исследуемом процессе отсутствуют, то применяется механизм поведенческого анализа.

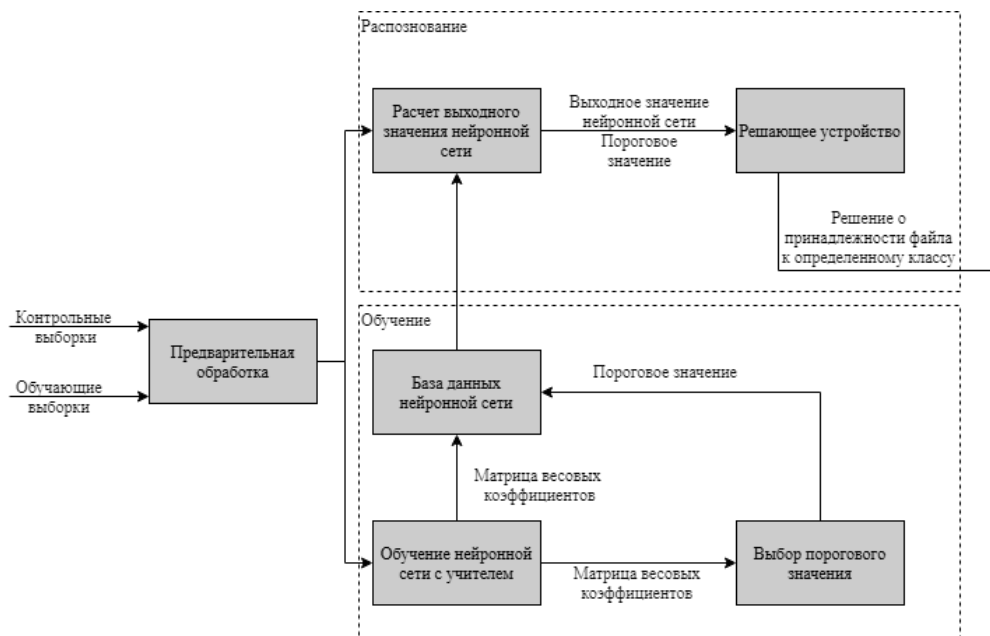


Рис. 1. Функциональная модель системы распознавания программного обеспечения удаленного управления с применением искусственной нейронной сети

В подсистеме предварительной обработки реализуются следующие функции: выделение цепочек событий компьютерной системы, для каждого анализируемого процесса; определение параметров этих событий, реализующих процедуры выделения цепочек событий компьютерной системы для анализируемого процесса,

определения параметров каждого перехваченного события, передачи определенных параметров перехваченных событий подсистеме обучения или распознавания.

Подсистема обучения, блок схема алгоритма которой представлена на рис. 2, предназначена для корректировки весовых коэффициентов связей, с целью повышения точности обучения нейронной сети. На вход подсистемы обучения поступает множество векторов сформированный системой предварительной обработки, а также априорно известные выходные сигналы для каждого вектора. При этом производится подстройка весовых коэффициентов с целью повышения точности классификации нейронной сети.

Изначала производится инициализация элементов матрицы весовых коэффициентов случайными значениями. Далее вычисляются выходные значения нейронной сети на основе величины входного вектора. Затем определяется разница между априорно известными и полученными значениями выхода нейронной сети. Если ожидаемый выход нейронной сети и полученный не совпадают, то производится корректировка весовых коэффициентов. В такой последовательности обрабатываются все вектора из обучающей выборки. Результатом является матрица весовых коэффициентов обученной нейронной сети.

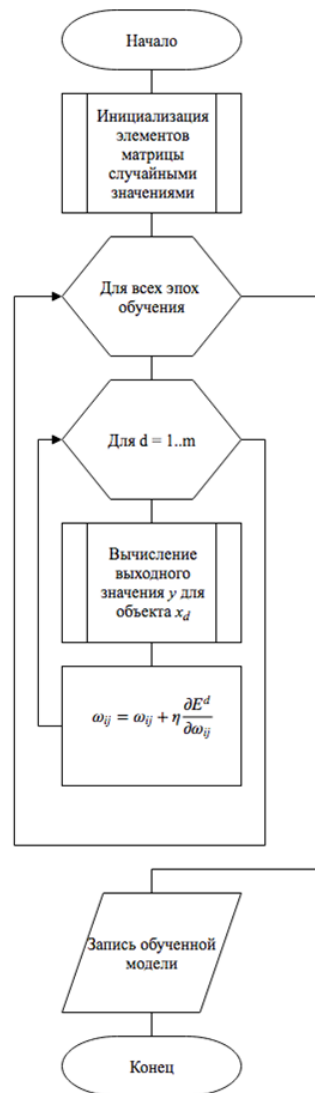


Рис. 2. Блок-схема алгоритма подсистемы обучения

Задача подсистемы распознавания заключается в принятии решения о наличии функционала, предназначенного для удаленного управления. Процедура распознавания включает в себя два основных этапа: 1) проведение расчета выходных значений нейронных сетей; 2) этап принятия решения о принадлежности файла к определенному классу. На первом этапе на вход системы распознавания поступает сформированная в подсистеме предварительной обработки анализируемого файла последовательность признаков. После этого производится расчет значений выходов нейронной сети с использованием матрицы весовых коэффициентов, занесенных в базу данных. На втором этапе производится сравнение полученного значения с пороговым, и принимается решение о принадлежности анализируемого файла к определенному классу программ.

На основе вышеупомянутого алгоритма разработано программное обеспечение, представляющее собой ИОС сканер, который осуществляет поиск в целевой системе различных признаков, указывающих на присутствие в операционной системе программного обеспечения предназначенного для удаленного управления.

Индикаторы компрометации описываются с использованием открытого стандарта OpenIOC [14]. Данный стандарт построен на базе XML и содержит свыше 500 различных индикаторов, преимущественно узловых, таких как, файл, драйвер, диск, процесс, реестр, система, хэш и т.п. Описание индикаторов компрометации с применением данного стандарта, позволяет использовать их кроме разработанного программного продукта в уже имеющихся системах SOC, SIEM различных производителей.

Сигнатурный поиск осуществляется с использованием YARA – инструмент, нацеленный на оказание помощи исследователям вредоносных программ для выявления и классификации образцов вредоносных программ.

V. ОБСУЖДЕНИЕ РЕЗУЛЬТАТОВ

При решении поставленной задачи для формирования признакового пространства была проведена оценка распределения событий в компьютерной системе по группам для различных классов программного обеспечения. Анализ полученных результатов позволил сделать вывод, что для программ удаленного управления, характерна высокая статистика вызовов системных функций связанных с передачей данных по сети, захватом изображения с экрана, эмуляция ввода с клавиатуры и мыши.

Анализ эффективности алгоритма разработанной модели осуществлялся с применением оценок вероятностей ошибок первого ($p_{ош1}$) и второго рода ($p_{ош2}$).

Расчет оценки вероятности ошибки первого рода производится в соответствии с выражениями:

$$p_{ош1} = \frac{N_{ош1}}{N_{cir}}; p_{ош2} = \frac{N_{ош2}}{N_{rat}},$$

где $N_{ош1}$ – количество «чистых» программ, распознанных как программы, имеющие функционал удаленного управления; N_{cir} – общее количество «чистых» программ; N_{rat} – общее количество программ удаленного управления.

Проведенные эксперименты позволили обосновать такие параметры системы распознавания на основе искусственной нейронной сети как количество эпох обучения, число элементов обучающей выборки, минимальную длину анализируемой цепочки событий компьютерной системы.

VI. ВЫВОДЫ И ЗАКЛЮЧЕНИЕ

В ходе выполнения представленной работы была разработана модель системы распознавания программного обеспечения, имеющего в своем составе функционал, предназначенный для удаленного управления, на основе искусственной нейронной сети.

В процессе верификации разработанной аналитической модели и оценки эффективности полученного решения были поставлены три группы экспериментов, анализ результатов которых позволяет обосновать следующие значения параметров системы распознавания программного обеспечения для удаленного управления на основе искусственной нейронной сети:

1. Количество эпох обучения – 700;
2. Количество элементов обучающей выборки – 640
3. Минимальная длина анализируемой цепочки событий компьютерной системы – 120.

Данные параметры являются достаточными для распознавания программного обеспечения, имеющего в своем составе функционал для удаленного управления с вероятностью ошибки первого рода менее 0,02 и вероятностью ошибки второго рода менее 0,015.

СПИСОК ЛИТЕРАТУРЫ

1. Атаки на промышленные предприятия с использованием RMS и TeamViewer. URL: <https://ics-cert.kaspersky.ru/reports/2018/08/01/attacks-on-industrial-enterprises-using-rms-and-teamviewer/> (дата обращения 12.05.2020).
2. Cobalt snatch. URL: <https://www.ptsecurity.com/analytics/Cobalt-Snatch-rus.pdf>, свободный (дата обращения 12.05.2020).
3. Fares A., Sharawy M. Intrusion Detection: Supervised Machine Learning // Journal of Computing Science and Engineering. 2011. Vol. 5 (4). P. 305–313.
3. Булахов Н. Г., Калайда В. Т. Методы обнаружения и обезвреживания саморазмножающихся вирусов // Доклады ТУСУРа. 2008. № 2 (18). С. 78–82.

4. Hajji H. Statistical analysis of network traffic for adaptive faults detection // IEEE Trans. Neural Networks. 2005. Vol. 16 (5). P. 1053–1063.
5. Hsu C. C., Huang Y. P. Incremental clustering of mixed data based on distance hierarchy // Expert Syst. Appl. 2008. Vol. 35 (3). P. 1177–1175.
6. Yu W. Y., Lee H. M. An incremental-learning method for supervised anomaly detection by cascading service classifier and its decision tree methods // Proceedings of the Pacific Asia Workshop on Intelligence and Security Informatics. 2009. P. 155–160.
7. Beghdad R. Critical study of neural networks in detecting intrusions // Computers and Security. 2008. Vol. 27 (5-6). P. 168–175.
8. Faraoun K. M., Boukelif A. Neural networks learning improvement using the K-means clustering algorithm to detect network // International Journal of Computational Intelligence. 2006. Vol. 3 (2). P. 161–168.
9. Wang G., Hao J., Mab, J. Huang L. A new approach to intrusion detection using Artificial Neural Networks and fuzzy clustering // Expert Systems with Applications. 2010. Vol. 37 (9). P. 6225–6232.
10. Fares A., Sharawy M. Intrusion Detection: Supervised Machine Learning // Journal of Computing Science and Engineering. 2011. Vol. 5 (4). P. 305–313.
11. Mukhopadhyay I., Chakraborty M., Chakrabarti S., Chatterjee T. Back propagation neural network approach to Intrusion Detection System // Recent Trends in Information Systems (ReTIS). 2011. P. 303–308.
12. Kayacik H. G., Zincir-Heywood A. N., Heywood M. I. A hierarchical SOM-based intrusion detection system // Engineering Applications of Artificial Intelligence. 2007. Vol. 20 (4). P. 439–451.
13. Hutchins E. M., Cloppert M. J., Amin R. M. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains // Leading Issues in Information Warfare & Security Research. 2011. Vol. 1, no. 1. 80 p.
14. Will Gibb, OpenIOC: Back to the Basics // FireEye Inc. URL: <https://www.fireeye.com/blog/threat-research/2013/10/openioc-basics.html> (дата обращения 12.05.2020).

УДК 004.056:159.9

ПАРАДОКСАЛЬНОЕ ЗВЕНО ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ

PARADOXIAL LINK IN INFORMATION SECURITY

Г. А. Дронова, В. Ю. Дронов

Новосибирский государственный технический университет, г. Новосибирск, Россия

G. A. Dronova, V. U. Dronov

Novosibirsk State University, Novosibirsk, Russia

Аннотация. Целью настоящей статьи является постановка вопроса, приглашение коллег к обсуждению проблематики, находящейся на стыке разных областей - информационной безопасности и психологии, и заключающейся в особенностях взаимодействия системы менеджмента информационной безопасности (ИБ) в ее социальном аспекте и человека, как элемента, создающего, сопровождающего, использующего эту систему. Основываясь на опыте внедрения систем менеджмента ИБ, с одной стороны, и знании психического устройства человека, с другой стороны, авторы исследуют человеческий фактор, как особую причину нарушений ИБ.

Ключевые слова: информационная безопасность, нарушения информационной безопасности, человеческий фактор, система менеджмента информационной безопасности.

DOI: 10.25206/2310-9793-8-2-104-108

1. ВВЕДЕНИЕ

И российские [1], и зарубежные [2] исследователи называют сотрудников, контрагентов компаний в качестве основной причины, вызывающей нарушения информационной безопасности (ИБ). Мировой лидер в области консалтинга и аудита - компания PricewaterhouseCoopers в своем обзоре "The Global State of Information Security Survey 2018. Singapore highlights." [3] именует персонал "ахиллесовой пятой безопасности", подчеркивая устойчивую уязвимость системы ИБ именно с этой стороны. И это – не новые свидетельства.

Самый успешный из известных хакеров современности Кевин Митник (англ. Kevin Mitnick) в 2002 году опубликовал книгу "Искусство обмана" (англ. "The Art of Deception"), обобщив в ней свой многолетний опыт искусно проведенных операций по проникновению в компьютерные сети и системы крупных организаций в 80-